

Embryo Presence Classifier 🥚 ➡ 📷

This notebook trains a **binary classifier** that predicts whether a given time-lapse frame from the *Human Embryo Time-Lapse* dataset contains a visible embryo (①) or is empty / out-of-focus (②).

We fine-tune a lightweight **ResNet-18** on a subset of embryos annotated by me.

Embryo ID	Empty frame ranges
AAL839-6	475-558
AB91-1	376-558
AC264-1	209-343
ADM715-1-2	358-446
AG782-8	375-565
AH988-4	471-551
AHS115-5	272-293 & 474-557
AHS599-4	459-464
ALR493-6	470-566
AM33-2	287-562
AM716-1	476-572
AM716-7	476-572
...	...

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Install / imports

```
# @title Install / imports
# !pip -q install torch torchvision torchaudio --extra-index-url https://download.pytorch.org/whl/cu118

from pathlib import Path
import re, random, itertools, math, json, time
import numpy as np
from PIL import Image
from tqdm.auto import tqdm

import torch, torch.nn as nn
torch.backends.cudnn.benchmark = True
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as T
import torchvision.transforms.v2 as v2
import torchvision.models as models
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report, confusion_matrix
```

Dataset locations

```
# @title Dataset locations
IMG_DIR = Path("/content/drive/MyDrive/Msc in AI/AI Applications/Human embryo time-lapse video dataset/embryo_dataset")
ANNOT_DIR = Path("/content/drive/MyDrive/Msc in AI/AI Applications/Human embryo time-lapse video dataset/embryo_dataset_annotator")

assert IMG_DIR.is_dir(), f"{IMG_DIR} not found"
assert ANNOT_DIR.is_dir(), f"{ANNOT_DIR} not found"
```

Helper functions

```
# @title Helper functions
def extract_index(fp):
    """Extracts RUN#### index from a frame filename."""
```

```
m = re.search(r"RUN(\d+)", fp.name)
return int(m.group(1)) if m else None
```

Training subset specification

```
# @title Training subset specification
NO_EMBRYO_RANGES = {
    "AAL839-6" : [(475, 558)],
    "AB91-1" : [(376, 558)],
    "AC264-1" : [(209, 343)],
    "ADM715-1-2" : [(358, 446)],
    "AG782-8" : [(375, 565)],
    "AH988-4" : [(471, 551)],
    "AHS115-5" : [(272, 293), (474, 557)],
    "AHS599-4" : [(459, 464)],
    #"ALR493-6" : [(470, 566)], Truncated images
    "AM33-2" : [(287, 562)],
    "AM716-1" : [(476, 572)],
    "AM716-7" : [(476, 572)],
    "AMT360-1-9" : [(390, 394)],
    "AS556-3" : [(473, 560)],
    "AS662-2" : [(275, 466)],
    "AS1015-2" : [(297, 371)],
    "BA518-7" : [(464, 553)],
    "BA560-1" : [(351, 380)], # Darkens in this range
    "BA560-2" : [(476, 574)],
    "BA782-2" : [(1, 1), (373, 376)],
    "BA1195-9" : [(256, 276), (377, 663)],
    "BC254-5" : [(381, 561)],
    "BC277-9" : [(366, 458)],
    "BC277-10" : [(179, 264)], # Darkens in this range
    "BC518-6" : [(419, 551)],
    "BC750-7" : [(465, 555)],
    "BC973-4" : [(473, 555)],
    "BE327-2" : [(168, 283), (473, 574)],
    "BE413-3" : [(353, 418)],
    "BE645-7" : [(375, 559)],
    "BF924-3" : [(471, 557)],
    "BG201-1" : [(191, 274)],
    "BG723-5" : [(377, 555)],
    "BH034-4" : [(374, 564)],
    "BI224-2" : [(357, 423)],
    #"BJ492-8" : [(145, 147)], Truncated Images
    "BJ497-7" : [(190, 460)],
    "BJ3371-9" : [(275, 413)]
}

EXCLUDE_FROM_TRAIN = {"AM365-7"} # blurred embryo - keep for later tests
TRAIN_EMBRYOS = sorted(NO_EMBRYO_RANGES.keys())
```

Dataset class

```
# @title Dataset class
class EmbryoPresenceDataset(Dataset):
    def __init__(self, img_dir, embryo_ids, no_embryo_ranges, transform=None):
        self.samples = [] # list of (path, label)
        self.transform = transform

        for eid in embryo_ids:
            folder = img_dir / eid
            if not folder.is_dir():
                print(f"⚠️ folder {folder} missing - skipped")
                continue

            # collect frame paths with index
            idx2path = {extract_index(fp): fp
                        for fp in folder.glob("*.jpg")}
            if extract_index(fp) is not None:

            # enumerate frames
            for idx, fp in idx2path.items():
                # decide label
```

```

        absent = any(s <= idx <= e for (s, e) in no_embryo_ranges[eid])
        label = 0 if absent else 1
        self.samples.append((fp, label))

    print(f"Dataset built with {len(self.samples):,} frames")

    def __len__(self): return len(self.samples)

    def __getitem__(self, i):
        path, label = self.samples[i]
        try:
            with Image.open(path) as im:
                im = im.convert("L") # 1-channel grayscale
                if self.transform: im = self.transform(im)
            return im, torch.tensor(label, dtype=torch.long)
        except OSError as e:
            print(f"⚠ Skipping truncated image: {path} - {e}")
            dummy_image = torch.zeros(1, self.transform.transforms[0].size[0], self.transform.transforms[0].size[1])
            return dummy_image, torch.tensor(label, dtype=torch.long)

```

Transforms & DataLoaders

```

# @title Transforms & DataLoaders
IMG_SIZE = 224
tfm = T.Compose([
    T.Resize((IMG_SIZE, IMG_SIZE)),
    T.ToTensor(),
    T.Normalize([0.5], [0.5]),
])

train_aug = v2.Compose([
    v2.RandomHorizontalFlip(p=0.5),
    v2.RandomVerticalFlip(p=0.5),
    v2.RandomRotation(45),
    v2.RandomResizedCrop(IMG_SIZE, scale=(0.8, 1.0),
                        ratio=(0.9, 1.1)),
    v2.GaussianBlur(kernel_size=(3, 3), sigma=(0.1, 2.0)),
    v2.RandomAdjustSharpness(1.5, p=0.3),
])

# --- embryo-level train / val / test split -----
import random

val_frac = 0.20 # 20 % of embryos
test_frac = 0.20 # 20 % of embryos

all_embryos = TRAIN_EMBRYOS.copy()
random.seed(42)
random.shuffle(all_embryos)

n_val = int(len(all_embryos) * val_frac)
n_test = int(len(all_embryos) * test_frac)

val_embryos = all_embryos[:n_val]
test_embryos = all_embryos[n_val:n_val + n_test]
train_embryos = all_embryos[n_val + n_test:]

print(f"Train embryos: {train_embryos}")
print(f"Val embryos: {val_embryos}")
print(f"Test embryos: {test_embryos}")

# build three **separate** datasets so no embryo appears in two splits
train_ds = EmbryoPresenceDataset(IMG_DIR, train_embryos, NO_EMBRYO_RANGES, transform=tfm)
val_ds = EmbryoPresenceDataset(IMG_DIR, val_embryos, NO_EMBRYO_RANGES, transform=tfm)
test_ds = EmbryoPresenceDataset(IMG_DIR, test_embryos, NO_EMBRYO_RANGES, transform=tfm)

BATCH_SIZE = 64
train_dl = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True, num_workers=0, pin_memory=True)
val_dl = DataLoader(val_ds, batch_size=BATCH_SIZE, shuffle=False, num_workers=0, pin_memory=True)
test_dl = DataLoader(test_ds, batch_size=BATCH_SIZE, shuffle=False, num_workers=0, pin_memory=True)
# -----

Train embryos: ['AM33-2', 'BC254-5', 'BA518-7', 'BJ497-7', 'AHS115-5', 'BC750-7', 'AAL839-6', 'BJ3371-9', 'AS556-3', 'BA560-2', 'AC
Val embryos: ['AM716-1', 'AS1015-2', 'AH988-4', 'BE413-3', 'BH034-4', 'BA782-2', 'BG201-1']
Test embryos: ['BC973-4', 'BC277-9', 'BE327-2', 'BG723-5', 'AM716-7', 'AMT360-1-9', 'BE645-7']
Dataset built with 11,413 frames

```

Dataset built with 3,408 frames
Dataset built with 3,662 frames

Model definition

```
# @title Model definition
device = "cuda" if torch.cuda.is_available() else "cpu"

model = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)
model.conv1 = nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3, bias=False) # adapt for 1-channel
model.fc = nn.Linear(model.fc.in_features, 2) # 2 classes
model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
```

Downloading: "<https://download.pytorch.org/models/resnet18-f37072fd.pth>" to /root/.cache/torch/hub/checkpoints/resnet18-f37072fd.pt
100%|██████████| 44.7M/44.7M [00:00<00:00, 136MB/s]

Training loop

```
# @title Training loop
import copy

EPOCHS = 50
best_val_loss = float("inf")
patience = 10 # epochs without improvement
epochs_no_improve = 0
best_model_wts = copy.deepcopy(model.state_dict())

for epoch in range(1, EPOCHS+1):
    # ---- train ----
    model.train()
    running_loss, correct = 0., 0
    for x, y in tqdm(train_dl, desc=f"Epoch {epoch}", leave=False):
        x = train_aug(x)
        x, y = x.to(device), y.to(device)

        optimizer.zero_grad()
        y_hat = model(x)
        loss = criterion(y_hat, y)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * x.size(0)
        correct += (y_hat.argmax(1) == y).sum().item()

    train_loss = running_loss / len(train_ds)
    train_acc = correct / len(train_ds)

    # ---- validate ----
    model.eval()
    running_vloss, val_correct = 0.0, 0
    with torch.no_grad():
        for x, y in val_dl:
            x, y = x.to(device), y.to(device)
            y_hat = model(x)
            vloss = criterion(y_hat, y)
            running_vloss += vloss.item() * x.size(0)
            val_correct += (y_hat.argmax(1) == y).sum().item()

    val_loss = running_vloss / len(val_ds)
    val_acc = val_correct / len(val_ds)
    # ----- early-stopping on val_loss -----
    if val_loss < best_val_loss - 1e-6: # small delta to avoid ties
        best_val_loss = val_loss
        epochs_no_improve = 0
        best_model_wts = copy.deepcopy(model.state_dict())
    else:
        epochs_no_improve += 1
```

```

        if epochs_no_improve >= patience:
            print(f"No val-loss improvement for {patience} epochs → stopping early.")
            break

    print(f"[{epoch}] loss {train_loss:.4f} | train_acc {train_acc:.3f} | val loss {val_loss:.4f} | val acc {val_acc:.3f}")

model.load_state_dict(best_model_wts)

```

```

[1] loss 0.0983 | train_acc 0.960 | val loss 0.0343 | val acc 0.994

[2] loss 0.0525 | train_acc 0.983 | val loss 0.1666 | val acc 0.890

[3] loss 0.0331 | train_acc 0.988 | val loss 0.0539 | val acc 0.993

[4] loss 0.0257 | train_acc 0.992 | val loss 0.1220 | val acc 0.925

[5] loss 0.0297 | train_acc 0.989 | val loss 0.0339 | val acc 0.992

[6] loss 0.0264 | train_acc 0.991 | val loss 0.1128 | val acc 0.934

```

final test evaluation

```

[7] loss 0.0267 | train_acc 0.991 | val loss 0.1335 | val acc 0.919

```

```

# @title final test evaluation
model.eval()
all_preds, all_true = [], []

for x, y in test_dl:
    logits = model(x.to(device))
    preds = logits.argmax(1).cpu()
    all_preds.extend(preds.numpy())
    all_true.extend(y.numpy())

test_acc = (np.array(all_preds) == np.array(all_true)).mean()
print(f"★ Test accuracy: {test_acc:.3f}")

print("\nClassification report (EMBRYO = 1, NO_EMBRYO = 0):")
print(classification_report(all_true, all_preds, target_names=["NO_EMBRYO", "EMBRYO"], digits=3))

```

```

★ Test accuracy: 0.999
[14] loss 0.0159 | train_acc 0.995 | val loss 0.0652 | val acc 0.974
Classification report (EMBRYO = 1, NO_EMBRYO = 0):
No val-loss improvement for 10 epochs → stopping early.
<All keys matched successfully>

```

	precision	recall	f1-score	support
NO_EMBRYO	1.000	0.995	0.998	859
EMBRYO	0.999	1.000	0.999	2803
accuracy			0.999	3662
macro avg	0.999	0.998	0.998	3662
weighted avg	0.999	0.999	0.999	3662

Save model + quick demo

```

# @title Save model + quick demo
model.eval()
MODEL_PATH = "embryo_presence_resnet18.pth"
torch.save(model.state_dict(), MODEL_PATH)
print("Model saved to", MODEL_PATH)

# Visualise a few predictions
idx_present = [i for i, (_, lbl) in enumerate(test_ds.samples) if lbl == 1]
idx_absent = [i for i, (_, lbl) in enumerate(test_ds.samples) if lbl == 0]

n_each = 10
sel_inds = random.sample(idx_present, n_each) + random.sample(idx_absent, n_each)
random.shuffle(sel_inds)

images = torch.stack([test_ds[i][0] for i in sel_inds])
labels = torch.tensor([test_ds[i][1] for i in sel_inds])

```

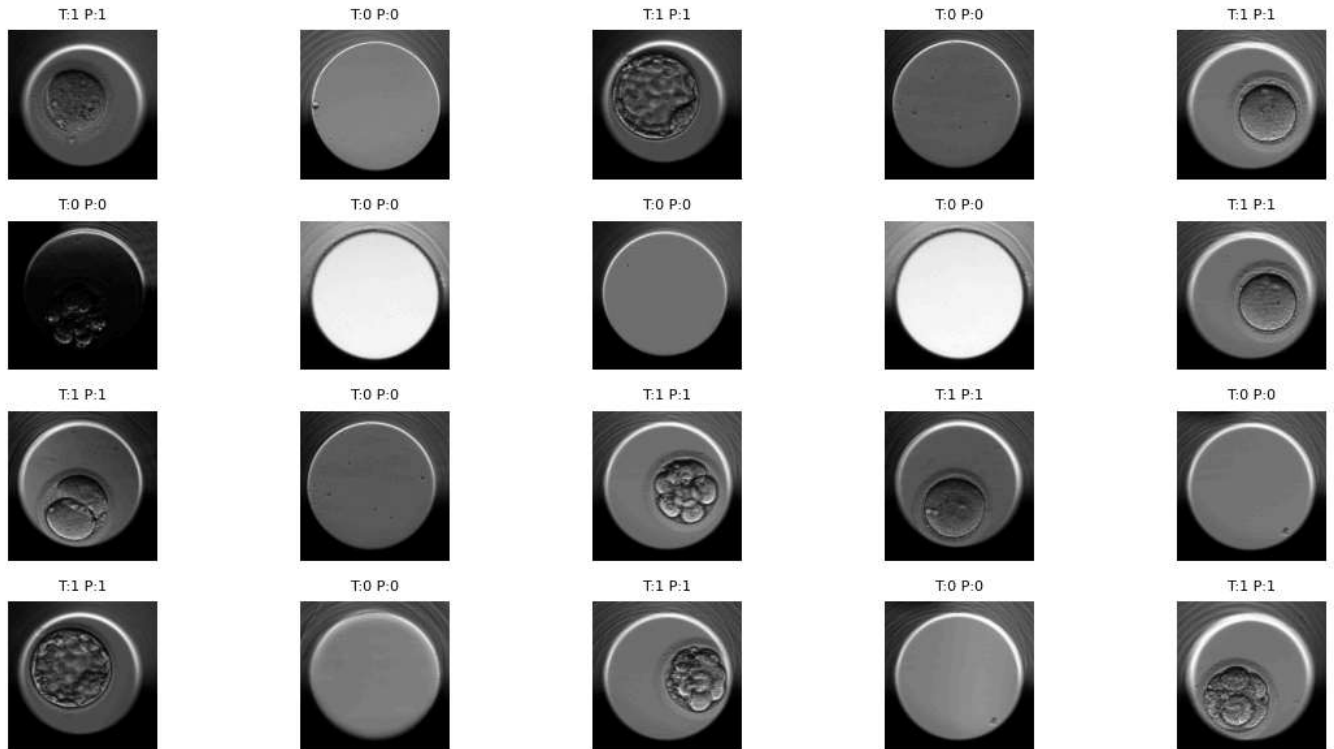
```

preds = model(images.to(device)).argmax(1).cpu()

fig, axes = plt.subplots(4, 5, figsize=(12, 6))
for ax, img, lab, pred in zip(axes.flatten(), images, labels, preds):
    ax.imshow(img.squeeze(0).numpy()*0.5 + 0.5, cmap="gray")
    ax.set_title(f"T:{lab.item()} P:{pred.item()}", fontsize=8)
    ax.axis("off")
plt.tight_layout()

```

Model saved to embryo_presence_resnet18.pth



```

# 1) Brightness-only baseline
abs_scores, pres_scores = [], []
for img, lbl in random.sample(test_ds.samples, 400): # 400 random test frames
    x = np.array(Image.open(img).convert("L"), dtype=np.float32)/255.
    score = x.mean()
    (pres_scores if lbl==1 else abs_scores).append(score)

tau = 0.15
preds = [0 if s<tau else 1 for s in abs_scores+pres_scores]
true = [0]*len(abs_scores) + [1]*len(pres_scores)
print("Brightness baseline acc:", (np.array(preds)==true).mean())

```

Brightness baseline acc: 0.7875

```

# 2) Saliency check: does the net look at the embryo?
!pip -q install torchcam # if not installed
# -----
# Grad-CAM montage: 50 % EMBRYO, 50 % NO_EMBRYO (test frames)
# -----
from torchcam.methods import GradCAM
import torch.nn.functional as F

model.eval() # keep weights frozen
cam_extractor = GradCAM(model, target_layer=model.layer4[-1])

# ----- 1. Balanced sampling -----
random.seed(0)

idx_embryo = [i for i, (_, lbl) in enumerate(test_ds.samples) if lbl == 1]
idx_empty = [i for i, (_, lbl) in enumerate(test_ds.samples) if lbl == 0]

n_each = 6

```

```

sel_inds = random.sample(idx_embryo, n_each) + random.sample(idx_empty, n_each)
random.shuffle(sel_inds)

# ----- 2. Compute CAMs -----
images, labels, cams = [], [], []
for idx in sel_inds:
    x, lbl = test_ds[idx]          # x: [C,H,W] tensor
    x_batch = x.unsqueeze(0).to(device)

    # Forward *with* gradients
    logits = model(x_batch)

    # CAM for the predicted class (or use lbl to see GT class)
    cls_id = logits.argmax(1).item()
    cam_small = cam_extractor(cls_id, scores=logits)[0]
    cam_small = cam_small.squeeze(0)

    # Upsample to input resolution
    cam_up = F.interpolate(cam_small.unsqueeze(0).unsqueeze(0),
                           size=(IMG_SIZE, IMG_SIZE),
                           mode='bilinear',
                           align_corners=False).squeeze()

    cam_up = (cam_up - cam_up.min()) / (cam_up.max() - cam_up.min() + 1e-6)

    images.append(x)
    labels.append(lbl)
    cams.append(cam_up.cpu())

# ----- 3. Plot grid -----
n_cols = 6
n_rows = math.ceil(len(images) / n_cols)
fig, axes = plt.subplots(n_rows, n_cols, figsize=(14, 2.5*n_rows))

for ax, img, lbl, cam in zip(axes.flatten(), images, labels, cams):
    base = img.squeeze()*0.5 + 0.5      # un-normalise to [0,1]
    ax.imshow(base.numpy(), cmap='gray')
    ax.imshow(cam.numpy(), cmap='jet', alpha=0.4)
    ax.set_title("EMBRYO" if lbl==1 else "NO_EMBRYO", fontsize=8,
                 color='lime' if lbl==1 else 'red')
    ax.axis('off')

plt.tight_layout()
plt.show()
# -----

```

```
61.0/61.0 kB 3.8 MB/s eta 0:00:00
46.0/46.0 kB 4.6 MB/s eta 0:00:00
18.3/18.3 MB 108.7 MB/s eta 0:00:00
363.4/363.4 MB 2.9 MB/s eta 0:00:00
13.8/13.8 MB 123.3 MB/s eta 0:00:00
24.6/24.6 MB 98.7 MB/s eta 0:00:00
883.7/883.7 kB 58.4 MB/s eta 0:00:00
664.8/664.8 MB 2.1 MB/s eta 0:00:00
211.5/211.5 MB 4.8 MB/s eta 0:00:00
56.3/56.3 MB 43.4 MB/s eta 0:00:00
127.9/127.9 MB 20.4 MB/s eta 0:00:00
207.5/207.5 MB 4.3 MB/s eta 0:00:00
21.1/21.1 MB 106.7 MB/s eta 0:00:00
```

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the so thinc 8.3.6 requires numpy<3.0.0,>=2.0.0, but you have numpy 1.26.4 which is incompatible.

